

Agentic Insurance RAG

Canadian insurance documents, clause-aware retrieval, and an agent that checks exclusions before it answers.

5-week build • same step structure as v1, components updated • supersedes section 6 of the New Grad Career Plan

Why insurance plus agentic beats either one alone

A generic “chat with your documents” build has to manufacture a reason for the agent to exist. Insurance hands you one for free: **a grant of coverage means nothing until you have also checked the exclusions, the endorsements, and the defined terms.** No single chunk contains the answer to a real coverage question.

That makes decomposition and multi-step retrieval a *domain requirement* rather than architecture fashion — which is exactly the distinction an interviewer is listening for. It also gives you a safety metric nobody else reports: how often the system found the coverage and missed the exclusion that voids it.

What this closes. All four gaps from the career plan — no public RAG project, no evaluation work, no tests, unbacked deployment claims — plus agentic orchestration and MCP, the fastest-moving keywords in current Toronto postings. One repository, six signals.

What changed from v1. The step structure, the five-week schedule, and the ship-at-week-3 rule are unchanged. The corpus is now a curated Canadian insurance set instead of user-supplied PDFs and URLs; chunking is clause-aware; the agent gains a coverage-specific control flow; and the eval harness gains two domain metrics. Everything else carries over intact.

01 The five decisions that carry the project

#	Decision	Why it differentiates
1	Clause-aware chunking, proven by ablation	Insurance documents carry explicit structure — numbered clauses, definitions, schedules, endorsements. Chunking on those boundaries instead of fixed token windows is measurable. Keep the uniform-chunker baseline row to prove it was worth doing.
2	A schema whose locator is a clause reference	Every chunk carries document, section, and clause identifier. That is what lets a citation say “Section 4.2(b)” rather than “chunk 187” — and in this domain a citation a human can verify is the whole product.
3	Hybrid retrieval, with a domain reason for BM25	Dense retrieval alone cannot reliably find “section 4.2(b)” or an exact defined term. Sparse search handles clause numbers and policy identifiers; dense handles paraphrased questions. This is the clearest justification for hybrid you will find in any domain.
4	A coverage-aware agent control flow	The agent must check exclusions and endorsements before answering a coverage question, and resolve defined terms before interpreting them. Domain-driven control flow, not a generic ReAct loop.

#	Decision	Why it differentiates
5	Two safety metrics nobody reports	Exclusion recall (of the questions where an exclusion applies, how often was it surfaced) and false-answer rate on out-of-policy questions. Both are measurable, both are unusual, and both demonstrate you thought about what happens when the system is wrong.

02 Architecture

Layer	Responsibility	Stack
Corpus	Curated Canadian insurance documents, versioned, with provenance metadata per document.	Git-tracked manifest, checksums
Ingestion	Layout-aware PDF extraction preserving clause structure and tables.	PyMuPDF or Docling
Chunking	Clause and section boundaries; definitions indexed separately.	Custom, plus tiktoken for token accounting
Storage	One Postgres holding chunk text, embeddings, and full-text index together.	Postgres 16, pgvector, tsvector + GIN
Retrieval	Dense and sparse candidates, RRF fusion, cross-encoder rerank, confidence scoring.	sentence-transformers, bge-reranker or Cohere Rerank
Agent	Decomposition, definition resolution, mandatory exclusion check, bounded retry, cited answer or refusal.	LangGraph + your choice of LLM API
Eval	Golden set, ablation runner, exclusion recall, refusal scoring, CI regression gate.	ragas, pytest, custom harness
Serving	API, minimal UI, tracing, pre-loaded demo scenarios.	FastAPI, React or plain HTML, LangSmith or custom trace log
Ops	Reproducible build, tested on every push, live URL on AWS.	Docker, GitHub Actions + OIDC, ECR, EC2 (Graviton), S3, CloudWatch

The schema — write this first

```
# the contract every ingested document must satisfy

class Chunk:
    chunk_id: str
    doc_id: str
    doc_type: Literal["policy", "endorsement", "bulletin", "guide"]
    chunk_role: Literal["coverage", "exclusion", "definition",
                        "condition", "schedule", "other"]
    locator: str # e.g. "OAP1 s.4.2(b)" — human-verifiable
    ordinal: int # position, for neighbour fetch
    defined_terms: list[str] # capitalised terms appearing here
    text: str
    token_count: int
    embedding: list[float]
```

chunk_role is the new field and it is the one that makes the agent possible. Tagging each chunk as coverage, exclusion, definition, or condition at ingest time is what lets the agent scope a search to exclusions only. Classify with rules first (heading text, clause numbering, trigger phrases), fall back to an LLM pass for the ambiguous remainder, and spot-check the labels by hand.

03 Corpus and ingestion

Where the documents come from

You need roughly 8–15 documents — enough for real multi-hop questions, small enough to hand-label a golden set. Aim for one coherent scenario set (say, Ontario auto plus home) rather than a scattered pile.

- **Regulator-published policy forms.** Standard provincial auto policy forms and their approved endorsements are published for public use and are the ideal backbone — real clause structure, publicly redistributable, and genuinely multi-hop.
- **Regulator bulletins and guidance** from provincial regulators and OSFI — these add a second document type with different structure.
- **Industry consumer guides** from bodies such as the Insurance Bureau of Canada — plain-language material that contrasts nicely with policy language.
- **Sample policy wordings** that insurers publish openly for consumers.

Check redistribution terms before you deploy publicly

Some insurer policy wordings are copyrighted even when they are freely downloadable. For a publicly deployed demo, lean on regulator forms, government publications, and material explicitly published for consumer use. Where you are unsure, index the document but serve only your own extracted citations and a link to the original rather than rehosting the file.

Record source URL, publisher, retrieval date, and licence note for every document in a committed manifest. It takes an hour and it is the kind of provenance discipline that reads as professional.

PDF ingestion requirements

- **Layout-aware extraction.** Policy documents are frequently two-column. A naive extract interleaves the columns and silently poisons every chunk downstream.
- **Clause numbering preserved** and parsed into the locator. This is the single most important extraction detail in the project.
- **Tables extracted intact** as structured markdown — coverage limit and deductible schedules are tables, and a table shredded across three chunks is unretrievable.
- **Definitions section indexed separately** so defined terms can be resolved directly rather than found by similarity search.
- **Scanned-page detection.** If a page yields near-zero text, flag it rather than indexing an empty chunk. OCR is out of scope; detecting the case is not.

If you keep a **fetch-by-URL** feature for regulator bulletins, use a **domain allowlist** rather than accepting arbitrary URLs. That removes the server-side request forgery surface almost entirely and is two lines of code. Document the choice in a short SECURITY.md alongside your prompt-injection notes — retrieved documents are untrusted input, and saying so in writing is a strong signal.

04 Chunking, storage, retrieval

Strategy	Detail	Rationale
Clause-aware (primary)	Split on numbered clause and section boundaries; tables kept whole; target 400–800 tokens with 15% overlap	The document already tells you where the semantic boundaries are. Cutting at arbitrary token counts throws that away.
Definition index (separate)	Each defined term stored as its own retrievable unit, keyed by term	Lets the agent resolve a term directly instead of hoping similarity search surfaces the definitions section.
Uniform baseline (for the ablation)	Fixed 512-token windows, no structure awareness	You need this row to prove the clause-aware work paid for itself.

Retrieval pipeline

```

STEP 1 candidate generation, in parallel
  query --> dense : pgvector cosine --> top-20
  query --> sparse : Postgres ts_rank --> top-20
                      (sparse is what finds "s.4.2(b)")

STEP 2 fuse and reorder
  reciprocal rank fusion --> cross-encoder rerank --> top-5

STEP 3 gate
  confidence --> above threshold ? proceed
              --> below           ? one retry, then refuse

```

Have the reranker answer ready. Why rerank rather than simply retrieve more? Bi-encoders score query and chunk independently and miss interaction effects; a cross-encoder reads both together and reorders on real relevance. Retrieving more without reranking widens the context and dilutes precision.

05 The agent — the part that is new

LangGraph, extending your mail-agent work rather than starting cold. Four tools, a domain-driven control flow, one bounded retry, an explicit refusal state.

Tool	Signature	Purpose
search_corpus	(query, role_filter=None, doc_filter=None, k=5)	Primary retrieval. <code>role_filter</code> is the important one — it scopes a search to exclusions or conditions only.
resolve_definition	(term)	Direct lookup against the definition index. Insurance policies redefine ordinary words, and the definition changes the answer.
fetch_neighbors	(chunk_id, window=1)	Pull adjacent clauses by ordinal when a hit lands mid-provision.
list_documents	()	Lets the agent scope its own search and answer “what do you have?”

Control flow — the coverage loop

This is the sequence that makes the project domain-specific rather than generic. Encode it in the graph; do not leave it to the model's discretion.

```

1. DECOMPOSE
  "burst pipe while away 3 weeks – am I covered?"
  --> is water damage a covered peril?
  --> is there a vacancy / unoccupancy exclusion?
  --> how is "vacant" defined, and over what period?

2. RESOLVE TERMS  resolve_definition("vacant")

3. FIND COVERAGE  search_corpus(role_filter="coverage")

4. MANDATORY EXCLUSION CHECK <-- never skipped
  search_corpus(role_filter="exclusion")
  search_corpus(role_filter="condition")

5. SYNTHESIZE  cite every clause; state what was checked
               low confidence ? one retry, then refuse

```

- **Step 4 is a graph edge, not a prompt instruction.** If the model can decide to skip the exclusion check, it eventually will. Making it structural is the design decision worth defending in an interview.
- **Bound everything.** One reformulation retry, hard cap on total steps — six is generous. Unbounded agent loops read as inexperience to anyone who has operated one.
- **State what was checked, not just what was found.** “Checked coverage s.3.1, exclusions s.4.2–4.7, conditions s.6” is far more trustworthy than a bare answer, and it costs nothing.
- **Refuse cleanly.** “This policy does not address that” must be a reachable, tested state.

The disclaimer is a product decision, not boilerplate

A system that answers coverage questions is one careless sentence away from reading as insurance advice. Handle it deliberately: never phrase an answer as a recommendation, always attach verifiable clause citations, surface uncertainty explicitly, and state plainly in the interface that this is a document retrieval tool and not advice.

Treat this as part of the engineering rather than a footer nobody reads. Being able to explain *why* you constrained the output format is a stronger signal than the retrieval work itself at most companies that would hire you for this.

06 The evaluation harness

Build the golden set before you write any retrieval code

Doing this in the wrong order quietly invalidates everything after it — write retrieval first and you will unconsciously author questions your implementation already answers.

Target: 40–60 question/answer pairs, each labelled with the clause(s) that should be retrieved. Split them three ways: **single-hop** (answer in one clause), **multi-hop** (coverage plus an exclusion or a definition — aim for at least a third of the set), and **unanswerable** (15–20 plausible, on-topic questions the corpus genuinely does not address).

The README headline table

Configuration	recall@5 single-hop	recall@5 multi-hop	exclusion recall	false-ans. rate	p95 latency
Dense only, uniform 512-token chunks					
+ clause-aware chunking					
+ BM25 hybrid (RRF)					
+ cross-encoder rerank					
+ agent with coverage loop					

The multi-hop column is the story. Expect naive retrieval to score near zero there and the agent row to jump. That gap is the entire justification for the agent, measured rather than asserted — and it is the single most persuasive number in the repository.

What each metric is for

- **recall@5, split single-hop versus multi-hop.** Retrieval-only, measured before any generation evaluation. An aggregate number hides that multi-hop is broken.
- **Exclusion recall.** Of the questions where an exclusion applies, how often did the system surface it? A confident “yes, covered” that missed the exclusion is the worst possible failure, and this is the metric that catches it.
- **False-answer rate** on the unanswerable set — how often it invents rather than refuses.

- **Groundedness, answer relevance, context precision** via ragas, once retrieval is settled.
- **p95 latency and cost per query** from your trace log. Cost awareness reads as production experience and takes an afternoon to instrument.

Wire the eval suite into CI as a regression gate. A pull request that drops recall or exclusion recall below threshold fails the build. That one line in your workflow is worth more than the rest of the CI config combined — it shows you understand that these systems regress silently.

07 Testing, CI, deployment

- **pytest** on clause-boundary parsing, the locator format, chunk_role classification, and RRF fusion maths.
- **Fixtures:** a few small committed PDFs so tests run without network access.
- **Golden-set regression test** that fails CI when retrieval quality drops.
- **Agent trace assertions:** given a coverage question, assert the exclusion check actually ran and the step count stayed within cap. This is the test that proves your central design decision holds.
- **Refusal test:** assert refusal on the unanswerable set rather than an answer.
- **Prompt-injection test:** a document containing adversarial instructions must not change agent behaviour. Retrieved text is untrusted input.
- **Dockerfile** building clean from scratch, plus docker-compose for app and Postgres.
- **GitHub Actions** running lint, tests, and evals on every push, badge in the README.
- **Live URL on AWS** — see section 8. Linked from the repo header and your portfolio.
- **Three pre-loaded scenarios** (a covered claim, an excluded claim, an unanswerable question) so a recruiter sees all three behaviours in ten seconds.

08 Deployment and ops on AWS

Worth doing, and for a specific reason: AWS, Docker, and CI/CD are all on your skills list with no artifact behind them. That is gap #4 from the career plan. Deploying this project on AWS converts three hollow claims into one true one you can talk about because you fought with it.

Two traps that will kill this before you notice

Trap 1 — the Free plan closes your account. Since July 2025, new AWS accounts pick a Free or Paid plan at signup. Both start with \$100 in credits, but **the Free plan expires after six months or when credits run out, whichever comes first, and the account then auto-closes.** Your live demo would die mid-job-search. Choose the **Paid plan** (or upgrade to it while credits are active — the remaining credits carry over) and control spend with a budget alarm instead.

Trap 2 — the NAT Gateway. Roughly \$32 a month before data processing charges. Put your instance in a private subnet behind one and your entire credit balance is gone in three months with nothing to show. **Use a public subnet with a tight security group.** For a single-box demo this is the correct architecture, not a compromise.

Double your credits on the way in

The \$100 you have can become \$200. AWS pays \$20 each for five onboarding activities: launching and terminating an EC2 instance, configuring an RDS database, deploying a Lambda function, testing a prompt in Bedrock, and

setting up a cost budget. You will do four of those anyway building this — do them deliberately and claim the credits. Note that credits expire twelve months after account creation, and they expire *immediately* if you join the account to an AWS Organization, so do not do that.

The architecture, and what it costs

One EC2 instance running docker-compose. Not glamorous, but it is what most small production systems actually look like, and the alternative (ECS Fargate + ALB + RDS) runs \$70–\$90 a month and would exhaust your credits in about six weeks.

Component	Choice	≈ monthly
Compute	EC2 t4g.small (ARM Graviton), single AZ, public subnet, docker-compose running app + Postgres/pgvector + Caddy for TLS	\$13
Public IPv4	One address — AWS charges for these now, even when attached to a running instance	\$4
Storage	EBS gp3, 20 GB	\$2
Registry	ECR for container images	< \$1
Object store	S3 for the document corpus and eval artifacts	< \$1
Logs	CloudWatch Logs for traces, 7-day retention set explicitly	\$1–2
LLM	Bedrock, or keep your existing provider	\$2–5
Total		≈ \$22–27

At that rate \$200 in credits covers roughly seven to eight months — which is the entire span of your job search. Verify current pricing for your region before you commit; these are estimates, not quotes.

Where the real learning is

- **GitHub Actions to ECR via OIDC role assumption**, not stored access keys. This is the highest-value thing in the section — short-lived federated credentials instead of long-lived secrets in your repo. Plenty of working engineers have never set it up, and it is a concrete security practice you can point at.
- **Security group as your only network boundary**. Ports 80 and 443 open to the world, 22 restricted to your IP, Postgres not exposed at all. Explaining why you did not need a NAT gateway is a better answer than having used one.
- **CloudWatch Logs Insights over your trace log**. Query p95 latency and cost per query directly from the logs. This is the ops half of the observability story your eval harness starts.
- **A budget alarm at \$25 and a hard one at \$50**. Set it before you deploy anything, not after.
- **Region: ca-central-1** keeps latency low for Toronto demos. Check that the Bedrock models you want are available there; if not, either call another region cross-region or stay with your current LLM provider. Keep every other resource in one AZ — cross-AZ traffic is billable and easy to incur by accident.

Keep embedding off the serving path

Ingestion and embedding are a **batch job**, not a request-path operation. Run them locally or on a temporary larger instance, write the vectors into Postgres, then terminate it. The always-on box then only has to embed the incoming query and run the cross-encoder, which fits comfortably in 2 GB.

This is both the correct architecture and the reason a \$13 instance is enough. If you hit memory pressure anyway, move to t4g.medium rather than reaching for a managed service.

Protect the ship date

AWS must not endanger week 3. The live URL on schedule matters more than which cloud it runs on. Budget one working day for the AWS deploy; if you burn through it fighting IAM or networking, ship to Fly.io or Railway in an hour, keep applying with that URL, and migrate to AWS in week 4 as a clean, unhurried exercise. Deploying twice is a small cost. Missing week 3 is not.

Write the cost table into your README. A portfolio project that documents its own running cost and the decisions taken to control it is unusual, and it reads as production awareness rather than tutorial completion. It also gives you a ready answer when someone asks what your system costs to operate — a question most junior candidates have never considered.

09 Five-week schedule — unchanged structure

Ship at week 3. Harden through week 5.

The step structure from v1 is intact. Five weeks invites scope creep, and scope creep is what kills these projects.

Ship a complete v1 at week 3, then deepen. A live URL you can attach to applications in week 3 beats a perfect system in week 5, because the application track runs the entire time.

Week	Build track (≈15 h)	Search track (≈8 h)
1	Schema and corpus manifest first. PDF ingestion with clause parsing. chunk_role classification, spot-checked. Golden set written before any retrieval code, with the multi-hop split. Postgres up in docker-compose.	Resume rewritten under new-grad framing. openRoles and skills fixed. 10 applications.
2	Dense baseline, measure. Clause-aware chunking, measure. BM25 + RRF, measure. Reranker, measure. Definition index. Every number recorded as you go.	12 applications. 5 referral attempts. Build-in-public post with the baseline numbers.
3	Ship week. Agent with the coverage loop, bounded retry, refusal. FastAPI + minimal UI. pytest, Dockerfile, GitHub Actions. Deploy — AWS if it goes smoothly, PaaS as the escape hatch. README with the eval table on top.	12 applications. Attach the live URL to every AI-role application from here on.

Week	Build track (≈15 h)	Search track (≈8 h)
4	Harden: ragas suite, exclusion-recall eval, tracing with latency and cost, CI regression gate, demo scenarios, SECURITY.md. AWS migration if week 3 used the escape hatch — ECR, OIDC deploy, CloudWatch, budget alarm. Retrieval failure taxonomy — read 50 misses by hand, categorise, fix the top two, show what moved. 90-second demo video.	12 applications. Two coffee chats. Light DSA maintenance.
5	Differentiate: wrap search_corpus as an MCP server . Then confidence calibration (reliability diagram, expected calibration error) and the long-context-versus-retrieval ablation if time allows.	12 applications. Write the eval findings up as a technical post.

On the MCP server in week 5: this folds the separate day 31–90 task from the career plan into this repository. Exposing corpus search as an MCP tool means one artifact evidences RAG, evaluation, testing, deployment, agents, *and* MCP — the six things current postings ask for.

10 Definition of done

v1 is done when all of these are true. Not before, and — importantly — not after. Anything beyond this list is scope creep unless it appears in week 5.

- ☐ A live URL a stranger can use in ten seconds without uploading anything.
- ☐ A README opening with the filled ablation table, not an architecture diagram.
- ☐ recall@5 reported separately for single-hop and multi-hop, with the uniform-chunker baseline visible.
- ☐ Exclusion recall and false-answer rate both reported.
- ☐ Every answer in the UI carries a verifiable clause citation.
- ☐ Passing CI badge with the eval regression gate active.
- ☐ A Dockerfile that builds clean on a machine that has never seen the project.
- ☐ Deployed on AWS, with the cost table and cost-control decisions in the README.
- ☐ CI pushing to ECR through an OIDC role — no long-lived AWS keys in the repo.
- ☐ A budget alarm set, and the account on the Paid plan so it will not auto-close.
- ☐ A test asserting the exclusion check ran, and a test asserting refusal.
- ☐ SECURITY.md covering prompt injection and, if applicable, the fetch allowlist.
- ☐ A 90-second demo video embedded in the README.
- ☐ Featured first in your portfolio, above the mail agent and GenDetect.

11 What this hands you in interviews

The question	Your answer comes from
“Tell me about a time your first approach failed.”	The uniform-chunker baseline and the near-zero multi-hop score. You measured it, categorised the misses by root cause, and fixed the top two. Before and after numbers on hand.

The question	Your answer comes from
“How do you know your AI system works?”	Retrieval-only evaluation before generation evaluation, split by hop count, plus a domain safety metric and a CI regression gate. This is the question that disqualifies most candidates.
“Why an agent here? Most RAG does not need one.”	Because coverage questions are multi-hop by construction, and you have the number showing single-pass retrieval fails them. The agent is a measured response to a measured failure, not architecture fashion.
“What could go wrong in production?”	Missed exclusions producing confidently wrong coverage answers, prompt injection through retrieved documents, silent retrieval regression on corpus updates. All three mitigated, all three testable.
“Why did you make that architecture choice?”	Cross-encoder over wider retrieval; one Postgres over a vector-DB plus search-engine split; the exclusion check as a graph edge rather than a prompt instruction. Three real trade-offs, each with a reason.

The framing to hold onto. Nobody is impressed that you built a retrieval system — that is a weekend for anyone with an API key. They are impressed that you *measured* one, found where it broke, fixed it, and can prove the fix. Every hour on the eval harness is worth three on features. When you are tempted to add a document type or polish the UI in week 4, go fill in a row of the table instead.